

Foundation Session 2

Mathematical Foundation for AI Systems

A beginner-friendly, in-depth journey through the math behind ML, Deep Learning, LLMs, RAG, and Agentic AI

Mr. Shridhar Galande



Session Overview

This session introduces the core mathematical foundations that power modern AI systems — from classical Machine Learning through Deep Learning, Large Language Models, Retrieval-Augmented Generation (RAG), and Agentic AI.

Every mathematical concept is explained from first principles, broken down term by term, paired with an intuitive analogy, and then applied to a business scenario, a real AI application, and a hands-on problem.

**Machine
Learning**

**Deep
Learning**

**Large Language
Models**

RAG

**Agentic
AI**



Learning Outcomes

- ✓ Understand how mathematics powers AI systems, from classical ML to modern GenAI
- ✓ Build deep intuition behind probability, conditional probability, and Bayes' Theorem
- ✓ Understand vectors, embeddings, and semantic space from first principles
- ✓ Perform basic matrix and tensor operations behind transformer models
- ✓ Interpret how LLMs use probability for next-token prediction
- ✓ Understand gradients and how AI systems learn from error
- ✓ Apply similarity metrics such as cosine similarity, used in RAG and semantic search
- ✓ Understand how AI agents use optimization and reward scores to make decisions



How We'll Learn

Every topic in this session is explored through four lenses:



Practical Intuition

Plain-language explanations, analogies, and formulas broken down term by term — nothing stays a black box.



Business-Oriented Understanding

Real workplace scenarios — marketing, loans, e-commerce, healthcare — where this math shows up.



AI Application Mapping

A direct line from the concept to the AI systems it powers — LLMs, RAG, recommendation engines, agents.



Hands-on Mathematical Reasoning

A "try it yourself" problem for every topic, so the math becomes a skill, not just a definition.



Session Agenda

PART 1 — FOUNDATIONS

- Why Math for AI
- AI Mathematics Mindset
- Probability Foundations
- Distributions & Sampling
- Vectors, Embeddings & Semantic Space

PART 2 — MATH BEHIND MODERN AI

- Matrices, Tensors & Transformer Operations
- Probability & Token Prediction
- Calculus, Gradients & Learning
- Similarity Metrics & Vector Search
- Optimization & AI Agent Decisioning
- Wrap-up + Hands-on Notebook Walkthrough

PART 1

Foundations of Mathematics for AI

Probability, Combinations, Vectors & Embeddings



AI Mathematics Mindset

PRACTICAL INTUITION

Thinking about AI like a Scientist, an Engineer, and a Decision Maker — all at once.

Scientist

Asks: what does the data actually show?

Engineer

Asks: how do I build something reliable with it?

Decision Maker

Asks: what's the best choice given the possibilities?

Permutations and combinations are the building blocks of this mindset — they let AI systems count and compare possibilities before choosing the best one.



AI Mathematics Mindset

DEEPER INTUITION

Think of it like this: picking a 3-person project team from 5 candidates. It doesn't matter who you name first — Alice-Bob-Carla is the same team as Carla-Alice-Bob. Combinations count groups, not orderings.

$${}^nC_r = n! / (r! \times (n-r) !)$$

- ✓ **$n!$** — all the ways to arrange every candidate in some order (a big number).
- ✓ **$r!$** — divides out the internal orderings of the r people you chose (since order within the team doesn't matter).
- ✓ **$(n-r) !$** — divides out the orderings of the people you didn't choose (they don't matter either).
- ✓ **Result** — what's left is purely the number of distinct groups, with all ordering noise removed.



Worked Example — Combinations

HANDS-ON MATHEMATICAL REASONING

Q. An ice-cream sundae with 3 flavors is made from a total of 5 available flavors. How many different sundaes are possible?

Order doesn't matter here — a sundae with chocolate-vanilla-mango is the same as mango-vanilla-chocolate — so we use **combinations**, not permutations.

$${}^5C_3 = 5! / (3! \times 2!) = 10$$

10 possible sundae combinations

AI link: AI systems constantly count and compare possibilities — from model configurations to search rankings.



Try It Yourself

A recommendation engine tests bundles of 2 products chosen from 4 items in stock. How many bundles are possible?

Answer: ${}^4C_2 = 4! / (2! \times 2!) = 6$ possible bundles.



Business Example — Marketing Channels

BUSINESS-ORIENTED UNDERSTANDING

Scenario: Choosing a Campaign Mix

A marketing team has 6 available channels (Email, Social, Search, Display, Influencer, SMS) and wants to run a campaign using exactly 3 of them at once.

$${}^6C_3 = 6! / (3! \times 3!) = 20$$

20 possible channel combinations to test

Why it matters: Before any AI can optimize a marketing budget, it first needs to know how many combinations exist to search through.



Try It Yourself

An HR team wants to shortlist 2 interview panels from 5 available senior staff. How many different panels are possible?

Answer: ${}^5C_2 = 5! / (2! \times 3!) = 10$ possible panels.



AI Application Mapping

WHERE THIS SHOWS UP IN AI

Counting and comparing possibilities is everywhere in AI systems:



Hyperparameter Search

Counting model configurations to test



Search Ranking

Comparing possible result orderings



Recommendation Bundles

Choosing product/content combinations



Probability Foundations

PRACTICAL INTUITION

Probability is the measure of how likely an event is to happen.

Probability

A number between 0 and 1 describing how likely something is.

Conditional Probability

The chance of an event, given that another has already happened.

Bayes' Theorem

Updates our belief about an event after seeing new evidence.

Why it matters: Spam filters, fraud detection, and recommendation engines all reason under uncertainty using exactly these ideas.



Probability Foundations

DEEPER INTUITION

Think of it like this: a detective updating a hunch. Before any evidence, you have a starting suspicion (the prior). A new clue arrives (the evidence). Bayes' Theorem tells you exactly how much to revise your suspicion — no more, no less.

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

- ✓ $P(A)$ — the prior: how likely A was before you saw any evidence.
- ✓ $P(B|A)$ — the likelihood: how likely the evidence is, assuming A is true.
- ✓ $P(B)$ — the total evidence: how likely you'd see this evidence under any scenario at all.
- ✓ $P(A|B)$ — the posterior: your updated belief about A, now that you've seen the evidence.



Worked Example — Bayes' Theorem

HANDS-ON MATHEMATICAL REASONING

Q. A diagnostic test is 85% accurate for people who have an illness, and 97% accurate for people who don't. About 1.2% of the population has the illness. If a person tests positive, what's the real chance they have it?

Out of 1,000 people...	
≈ 12 people actually have the illness	≈ 988 people do not have it
10 test positive (85% of 12)	≈ 30 test positive anyway (3% of 988)
Total testing positive ≈ 10 + 30 = 40	

Real chance of illness
given a positive test:

$$10 / 40 = 25\%$$



Try It Yourself

If the illness rate were 5% instead of 1.2% (all else equal), would the real chance of illness after a positive test go up or down? Why?

Answer: Up — a higher base rate means more true positives relative to false positives, so positive tests become more meaningful.



Business Example — Spam Detection

BUSINESS-ORIENTED UNDERSTANDING

Q. What is the probability an email is spam, given it contains the word "offer"?

Data: 50% of all emails are spam. 9% of spam emails contain "offer". 0.4% of genuine (ham) emails contain "offer".

$$P(\text{Spam} | \text{"offer"}) = (0.09 \times 0.5) / (0.09 \times 0.5 + 0.004 \times 0.5) \approx 0.957$$

≈ 96% chance the email is spam

Business impact: This is the exact math behind Naive Bayes spam filters that protect corporate inboxes at scale.



Try It Yourself

If "offer" appeared in 20% of spam emails (instead of 9%), would the probability of spam go up or down?

Answer: Up — a word more common in spam and rare in ham pushes the posterior probability of spam higher.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Fraud Detection

Flagging unusual transactions by probability



Recommendation Engines

Ranking items by likelihood of interest



Self-Driving Systems

Estimating likelihood of obstacles/events



Chatbots & Assistants

Weighing likely user intents



Medical AI

Diagnosis probability given symptoms/tests



Risk Scoring

Credit, insurance, and churn probability models



Hands-on Practice — Probability

HANDS-ON MATHEMATICAL REASONING

Q. A bank's model predicts loan default. Historically, 4% of applicants default. The model correctly flags 90% of defaulters, and incorrectly flags 6% of non-defaulters as risky. If an applicant is flagged as risky, what's the real chance they will actually default?

Hint: use natural frequencies — imagine 1,000 applicants and count step by step, just like the medical test example.

Answer: Out of 1,000 applicants: ≈ 40 default $\rightarrow 36$ flagged correctly. ≈ 960 don't default $\rightarrow \approx 58$ flagged anyway. Total flagged ≈ 94 . Real default chance given flagged $\approx 36/94 \approx 38\%$.



Distributions & Sampling

PRACTICAL INTUITION

Binomial Distribution

Models the number of "successes" out of a fixed number of yes/no trials.

Poisson Distribution

Models how many times a rare event happens in a fixed time window.

Why sample at all? AI systems rarely see the whole population — sampling lets us estimate patterns from a manageable subset of data.

Why it matters: These two distributions alone explain most "how many will happen" questions AI systems are asked to answer.



Distributions & Sampling

DEEPER INTUITION

Think of it like this: flipping a weighted coin several times (binomial) versus counting rare pings on a Geiger counter over an hour (Poisson). One counts successes out of known tries; the other counts rare events with no fixed number of "tries" at all.

Binomial: $P(X = k) = {}^nC_k p^k (1 - p)^{n-k}$

Poisson: $P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$

- ✓ nC_k — the number of different orders in which k successes could occur among n trials.
- ✓ $p^k (1 - p)^{n-k}$ — the probability of one specific arrangement of k successes and $(n-k)$ failures.
- ✓ λ (**lambda**) — the average rate of a rare event (e.g., 2 machines down on average).
- ✓ $\frac{e^{-\lambda} \lambda^k}{k!}$ — the probability of observing exactly k events, given that average rate.



Business Example — Customer Satisfaction

BUSINESS-ORIENTED UNDERSTANDING

Scenario: Metro Rail Customer Survey

A metro rail company surveys 8 senior citizens about new safety features. Each response is Yes/No, with a 60% historical "Yes" rate. What's the probability exactly 3 out of 8 say Yes?

$$P(X=3) = {}^8C_3 (0.6)^3 (0.4)^5 \approx 0.124$$

Business impact: This lets a business quantify how confident it can be in survey results before rolling out changes company-wide.



Try It Yourself

Using the same survey, what's the probability that all 8 people say Yes?

Answer: $P(X=8) = (0.6)^8 \approx 0.017$ — about a 1.7% chance.



Worked Example — Poisson Distribution

HANDS-ON MATHEMATICAL REASONING

Q. A subway station averages 2 ticket machines out of operation at any time. What's the probability exactly 3 machines are down right now?

$$P(X=3) = e^{-2} (2^3)/3! \approx 0.180$$

AI link: Poisson models are used for staffing predictions, server error-rate monitoring, and equipment failure forecasting.



Try It Yourself

Using the same station (average = 2 machines down), what's the probability that zero machines are down?

Answer: $P(X=0) = e^{-2} (2^0)/0! = e^{-2} \approx 0.135$ — about 13.5%.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Anomaly Detection

Flagging counts that deviate from expected distributions



Churn Modeling

Estimating how many customers will leave



Demand Forecasting

Predicting order volumes and inventory needs



Call Center Staffing

Poisson models for expected call volume



Hands-on Practice — Distributions

HANDS-ON MATHEMATICAL REASONING

Q. A customer support line receives an average of 3 escalations per hour (Poisson). What is the probability of exactly 2 escalations in the next hour?

Answer: $P(X=2) = e^{-3}(3^2)/2! \approx 0.224$ — about a 22% chance of exactly 2 escalations.



Vectors, Embeddings & Semantic Space

PRACTICAL INTUITION

- 1 **Vector:** A list of numbers that represents information mathematically. E.g., $[0.2, 0.8, -0.1]$
- 2 **Embedding:** A special vector that captures the meaning of a word, sentence, or image.
- 3 **Semantic Space:** A mathematical space where embeddings are positioned based on meaning.

Why it matters: Once meaning becomes numbers, computers can measure, compare, and search through it mathematically.



Vectors, Embeddings & Semantic Space

DEEPER INTUITION

Think of it like this: GPS coordinates, but for meaning instead of geography. Just as [latitude, longitude] pinpoints a location on Earth, an embedding's numbers pinpoint a concept's location in "meaning-space" — nearby coordinates mean related meaning.

$$\text{vector} = [\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_n]$$

$$\text{distance} = \sqrt{(\sum (\mathbf{x}_i - \mathbf{y}_i)^2)}$$

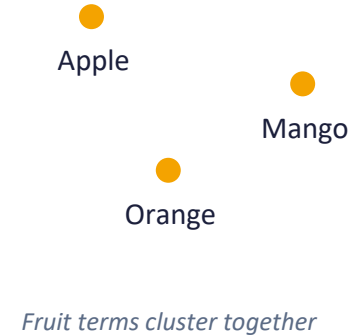
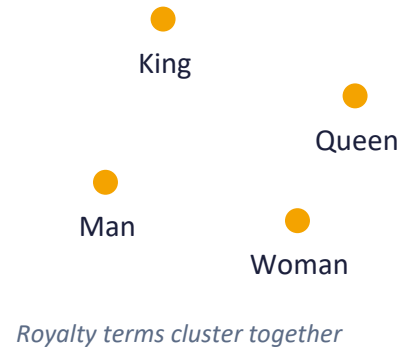
- ✓ $\mathbf{x}_1, \mathbf{x}_2, \dots$ — each number captures one learned aspect of meaning (not human-labeled, but discovered during training).
- ✓ n (**dimensions**) — real embeddings often use hundreds of dimensions to capture nuance; our examples use just 2 for visualization.
- ✓ **distance** — the straight-line gap between two vectors; smaller distance means more similar meaning.
- ✓ **semantic space** — the overall map these vectors live in, where clusters naturally form around related concepts.



Worked Example — Toy Semantic Map

HANDS-ON MATHEMATICAL REASONING

Words plotted by meaning — similar concepts cluster together, unrelated ones stay apart:



Try It Yourself

If "Princess" were added to this map, which cluster would it likely appear near, and why?

Answer: Near King/Queen — it shares royalty-related meaning, so its embedding would sit close to that cluster in semantic space.



Business Example — Product Recommendations

BUSINESS-ORIENTED UNDERSTANDING

Scenario: "Customers Also Bought"

An e-commerce platform converts each product's description, category, and reviews into an embedding vector. Products with nearby embeddings are shown as recommendations — a running shoe's embedding sits close to running socks, far from office chairs.

Business impact: Better embeddings directly translate to better recommendations — and measurably higher average order value.



Try It Yourself

Would you expect the embedding for "laptop bag" to be closer to "laptop" or to "garden hose"? Why?

Answer: Closer to "laptop" — they're semantically related and frequently bought together, so their embeddings would be positioned nearby.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Semantic Search

Finding results by meaning, not just keywords



Machine Translation

Mapping meaning across languages



Recommendation Systems

Matching users and items by embedding distance



RAG Retrieval

Fetching the most relevant document for a query



Fraud/Duplicate Detection

Clustering similar patterns together



Hands-on Practice — Vectors

HANDS-ON MATHEMATICAL REASONING

Q. Two customer profiles are represented as vectors: $A = [2, 3]$ and $B = [5, 7]$. What is the straight-line (Euclidean) distance between them?

Answer: distance = $\sqrt{((5-2)^2 + (7-3)^2)} = \sqrt{(9+16)} = \sqrt{25} = 5$. Smaller distance means more similar profiles.



BREAK

We'll resume with Matrices, Transformers, and how LLMs actually predict text.

PART 2

Math Behind Modern AI Systems

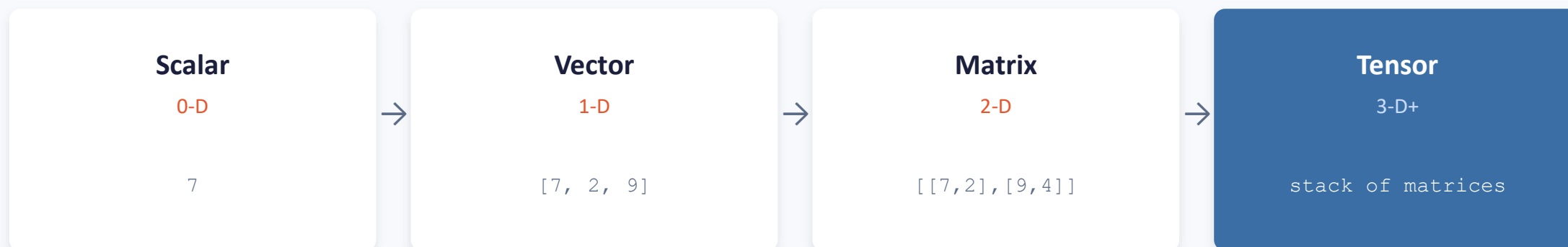
Matrices, Transformers, Gradients, Similarity & Optimization



Matrices, Tensors & Transformer Operations

PRACTICAL INTUITION

A matrix is a rectangular grid of numbers — essentially a collection of vectors.



Transformers — the architecture behind ChatGPT, Gemini, Claude, and Copilot — run almost entirely on matrix and tensor operations.



Matrices, Tensors & Transformer Operations

DEEPER INTUITION

Think of it like this: a spreadsheet formula that combines several weighted columns into one score — except a matrix multiplication does that for every row, against every output column, all at once.

$$(\mathbf{A} \times \mathbf{B})_{i j} = \sum_k \mathbf{A}_{i k} \times \mathbf{B}_{k j}$$

- ✓ $\mathbf{A}_{i k}$ — the single entry in row i , column k of matrix A (e.g., one applicant's score on one feature).
- ✓ $\mathbf{B}_{k j}$ — the single entry in row k , column j of matrix B (e.g., one feature's weight).
- ✓ $\sum_k$ (**sum over k**) — multiply each matching pair of entries together and add them up — this is a weighted total.
- ✓ $(\mathbf{A} \times \mathbf{B})_{i j}$ — the result: one combined number per row/column pair, repeated for the whole grid.



Business Example — Weighted Decision Scoring

BUSINESS-ORIENTED UNDERSTANDING

Scenario: Loan Approval Scoring

An AI system scores applicants using Income, Credit, and Repayment History, each with a different importance weight. This is exactly a matrix (applicants $\times$ features) multiplied by a weight vector.

$$\text{Score} = 0.5(\text{Income}) + 0.3(\text{Credit}) + 0.2(\text{Repayment})$$

Business impact: This single matrix operation scores thousands of applicants instantly and consistently.



Try It Yourself

If Income=80, Credit=70, Repayment=90, what's the applicant's score using the weights above?

Answer: $\text{Score} = 0.5(80) + 0.3(70) + 0.2(90) = 40 + 21 + 18 = 79$.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Attention Mechanism

Matrix multiplications compare every word to every other word



Image Processing

Images are tensors of pixel values



Batch Processing

Tensors let GPUs process many inputs at once



Hands-on Practice — Matrices

HANDS-ON MATHEMATICAL REASONING

Q. Multiply the following 2×2 matrices:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

(Multiply row-by-column, then sum)

Answer: $\begin{bmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

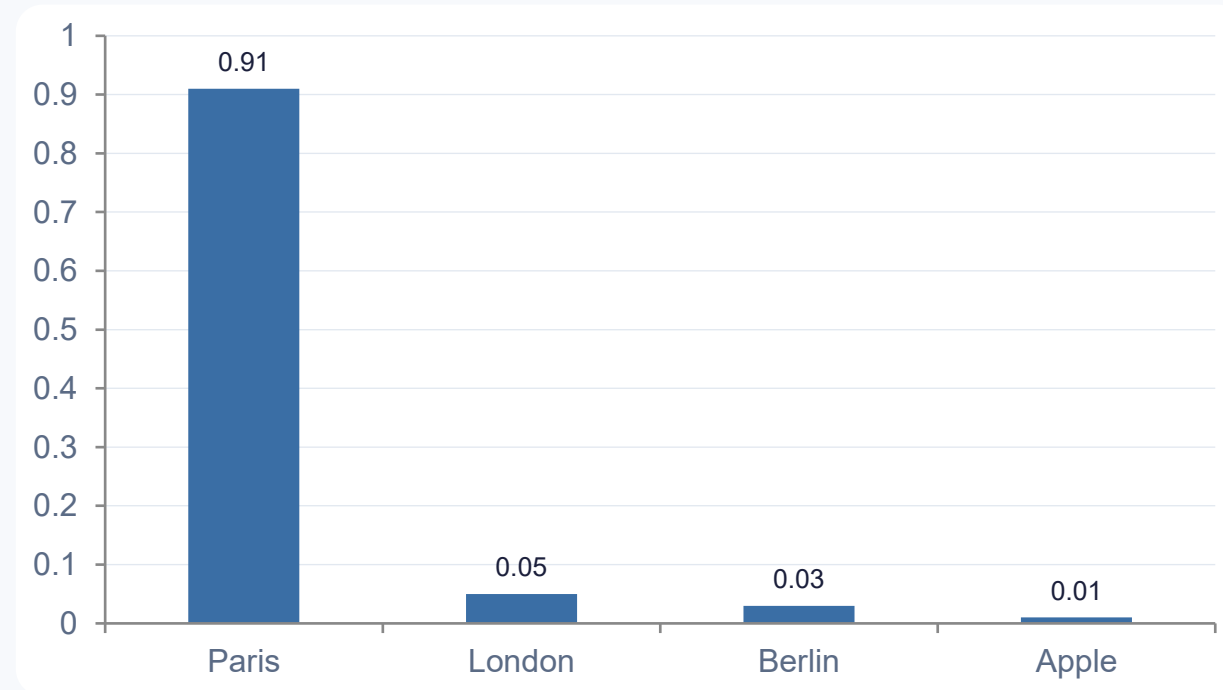


Probability & Token Prediction

PRACTICAL INTUITION

LLMs fundamentally work as **next-token probability prediction engines**.

Prompt: "The capital of France is ____"



Why "Paris" wins

The model has seen this fact millions of times during training, so it assigns "Paris" the highest probability (0.91) — this is next-token prediction in action.

Low temperature → picks the highest-probability token more consistently.



Probability & Token Prediction

DEEPER INTUITION

Think of it like this: a panel of judges giving raw scores to contestants, which are then converted into percentages of votes. The softmax function takes the model's raw "scores" for every possible next word and turns them into a clean probability distribution that sums to 100%.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}}$$

- ✓ z_i — the model's raw, unnormalized score for token i (can be any positive or negative number).
- ✓ e^{z_i} — exponentiating makes every score positive and amplifies the gap between high and low scores.
- ✓ $\sum_j e^{z_j}$ — the sum across all possible tokens, used to normalize everything into proper probabilities.
- ✓ **Temperature** — a dial that scales z_i before softmax: lower temperature sharpens the top choice, higher spreads probability more evenly.



Business Example — AI Medical Assistant

BUSINESS-ORIENTED UNDERSTANDING

Prompt: "Patient has high fever and persistent cough."

Token	Probability
Infection	0.58
Pneumonia	0.27
Allergy	0.12
Fracture	0.03

Why this matters for business: In high-stakes domains like healthcare, the model's probability distribution — not just its top answer — informs how much a human should double-check the suggestion.



Try It Yourself

Given the table above, if the assistant used a very low temperature, which diagnosis would it state most confidently?

Answer: "Infection" — the highest-probability token (0.58) is selected most consistently at low temperature.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Chatbots & Assistants

Every reply is a chain of next-token predictions



Code Completion

Predicting the most likely next line of code



Machine Translation

Predicting the most likely word in the target language



Temperature Control

Tuning randomness for creative vs. factual tasks



Hands-on Practice — Token Prediction

HANDS-ON MATHEMATICAL REASONING

Q. A customer-support AI predicts the next word after "Your refund will be processed within" with these probabilities: "3 days" = 0.62, "a week" = 0.25, "an hour" = 0.09, "a month" = 0.04. Which token does the model output at low temperature, and what does that probability tell you about the model's confidence?

Answer: "3 days" (0.62) — the model is fairly confident, but not certain; there's meaningful probability mass on alternatives.



Calculus, Gradients & Learning

PRACTICAL INTUITION

Gradients tell an AI model which direction to adjust its parameters to reduce error — this is how models learn from mistakes.

Worked Example — Training a Tiny Healthcare Model

Model: $y = w \cdot x$ (predicting risk score from one input)

Given $w = 2, \quad x = 3, \quad \text{actual } y = 8$

1. Predict $\hat{y} = w \times x = 2 \times 3 = 6$

2. Error $\text{actual} - \text{predicted} = 8 - 6 = 2$

3. Gradient step increase w slightly (e.g., $w: 2 \rightarrow 2.2$) to reduce error next time



Calculus, Gradients & Learning

DEEPER INTUITION

Think of it like this: walking down a foggy hill blindfolded. You can't see the bottom, but you can feel which direction slopes downward under your feet. You take a small step that way, feel again, and repeat — that's exactly what gradient descent does with error instead of altitude.

$$w_{\text{new}} = w_{\text{old}} - \text{learning_rate} \times (\partial \text{Loss} / \partial w)$$

- ✓ **Loss** — a number measuring how wrong the model's prediction is (e.g., squared error).
- ✓ **$\partial \text{Loss} / \partial w$** — the gradient: how much the loss would change if we nudged w slightly — the "slope you feel underfoot."
- ✓ **learning_rate** — how big a step to take each time; too large overshoots, too small learns painfully slowly.
- ✓ **w_{new}** — the updated parameter, moved a small step in the downhill (error-reducing) direction.



Business Example — Reducing Forecast Error

BUSINESS-ORIENTED UNDERSTANDING

Scenario: House-Price AI Model

A house-price model predicts \$130K for a house that actually sold for \$100K. The model must reduce this error before its next prediction.

$$\text{Error} = \text{Actual} - \text{Predicted} = 100 - 130 = -30$$

Business impact: Gradient descent uses this error to nudge the model's parameters, shrinking the error a little more with every training round.



Try It Yourself

If a sales-forecast model predicts 220 units but actual sales were 180, what's the prediction error?

Answer: Error = $180 - 220 = -40$ units (the model over-predicted by 40).



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Neural Network Training

Every layer's weights update via gradients



LLM Fine-Tuning

Adjusting a pretrained model to a new task



Recommendation Updates

Continuously improving predictions from feedback



Hands-on Practice — Gradients

HANDS-ON MATHEMATICAL REASONING

Q. A model predicts $y = w \cdot x$ with $w = 3$, $x = 4$. The actual value is $y = 20$. Calculate the prediction and the error. Should w increase or decrease next?

Answer: Prediction = $3 \times 4 = 12$. Error = $20 - 12 = 8$ (under-predicting). w should increase, since a larger w moves the prediction closer to 20.



Similarity Metrics & Vector Search

DEEPER INTUITION

Think of it like this: comparing compass directions instead of distances traveled. Two hikers could walk very different distances but still be heading the exact same direction. Cosine similarity ignores "how far" (magnitude) and measures only "which way" (direction).

$$\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \times \|\mathbf{B}\|)$$

- ✓ $\mathbf{A} \cdot \mathbf{B}$ (dot product) — multiply matching components of A and B and sum them; large when vectors point similarly.
- ✓ $\|\mathbf{A}\|, \|\mathbf{B}\|$ (magnitude) — the length of each vector; dividing by these removes the effect of size, leaving only direction.
- ✓ $\cos(\theta)$ — ranges from -1 (opposite direction) to 1 (identical direction); 0 means unrelated (perpendicular).
- ✓ **Why not just distance?** — two documents can differ in length (magnitude) but discuss the exact same topic (direction) — cosine similarity captures that correctly.



Similarity Metrics & Vector Search

PRACTICAL INTUITION

Cosine similarity measures how closely two vectors point in the same direction — the core mechanism behind semantic search and RAG.

Worked Example — Healthcare Semantic Search

Doctor searches: "heart attack treatment" → encoded as **Query** = [0.85, 0.80, 0.10]

Document	Vector	Similarity
D1: Cardiac arrest emergency care	[0.90, 0.80, 0.10]	1.00 ✓
D2: Diabetes diet management	[0.10, 0.20, 0.90]	0.31
D3: Flu vaccination schedule	[0.20, 0.10, 0.70]	0.37

Let's actually calculate D1's score (instead of just stating it):

1. Dot product: $Q \cdot D1 = (0.85 \times 0.90) + (0.80 \times 0.80) + (0.10 \times 0.10) = 0.765 + 0.640 + 0.010 = 1.415$

2. Magnitudes: $\|Q\| = \sqrt{0.85^2 + 0.80^2 + 0.10^2} \approx 1.172$ $\|D1\| = \sqrt{0.90^2 + 0.80^2 + 0.10^2} \approx 1.208$

3. Divide: $\cos(\theta) = 1.415 / (1.172 \times 1.208) = 1.415 / 1.416 = 0.999 \approx 1.00$

This confirms **D1 is the correct retrieval** — its direction is nearly identical to the query's.



Business Example — Product Similarity

BUSINESS-ORIENTED UNDERSTANDING

Scenario: "Similar Items" on a Retail Site

A shopper views a red running shoe. Two candidate products have embeddings $[0.9, 0.1]$ (blue running shoe) and $[0.1, 0.9]$ (leather office shoe), compared to the viewed shoe's embedding $[0.85, 0.15]$.

Business impact: The blue running shoe has much higher cosine similarity, so it's shown first — driving more relevant cross-sells and higher conversion.



Try It Yourself

Two article embeddings are $[1, 0]$ and $[0, 1]$. What does a cosine similarity of 0 tell you about these articles?

Answer: The vectors are orthogonal (perpendicular) — the articles share essentially no topical similarity.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



RAG Retrieval

Fetching the closest matching document for a query



Semantic Search

Ranking results by meaning-based closeness



Plagiarism / Duplicate Detection

Flagging near-identical content



Chatbot Memory

Recalling the most relevant past conversation



Hands-on Practice — Similarity

HANDS-ON MATHEMATICAL REASONING

Q. Two short sentence embeddings are $A = [1, 2]$ and $B = [2, 4]$. What do you notice about their direction, and what would you expect their cosine similarity to be?

Answer: B is exactly $2 \times A$, so they point in the identical direction. Cosine similarity = 1.0 — perfectly similar, even though the vectors have different magnitudes.



Optimization & AI Agent Decisioning

PRACTICAL INTUITION

Modern Agentic AI systems fundamentally work as optimization-driven decision systems — choosing the action that maximizes expected success.

Example — Choosing the Right Tool



An Agentic AI assistant is asked to "Generate quarterly financial analysis." It scores each available tool by expected usefulness (utility) and picks the highest-scoring one — Database Query, at 0.92.

Why it matters: This scoring-and-selecting pattern is the mathematical core of every AI agent's decision loop.



Optimization & AI Agent Decisioning

DEEPER INTUITION

Think of it like this: choosing a restaurant by weighing price, distance, and rating together into one score, then picking whichever restaurant scores highest. An AI agent does exactly this — except it repeats the choice thousands of times, refining its scoring based on which past choices led to good outcomes.

$\text{action}^* = \text{argmax}_a \text{Utility}(a)$

$\text{Return} = \Sigma \text{ rewards over time}$

- ✓ **Utility(a)** — a single score summarizing how good action a is expected to be, given the current situation.
- ✓ **argmax_a** — "the action a that makes $\text{Utility}(a)$ largest"; the agent doesn't average or guess, it picks the best-scoring option.
- ✓ **reward** — the feedback signal an agent receives after acting (positive for good outcomes, negative for bad ones).
- ✓ **Return ($\Sigma \text{ rewards}$)** — the cumulative reward over an entire sequence of actions, which reinforcement learning trains an agent to maximize.



Business Example — Reward Optimization

BUSINESS-ORIENTED UNDERSTANDING

Scenario: An autonomous research agent is rewarded for good retrievals and penalized for mistakes.

Action	Reward
Correct retrieval	+15
Incorrect retrieval	-8
Infinite loop	-25

The agent performs 2 correct retrievals and 1 incorrect retrieval:

$$\text{Total reward} = 2(15) - 8 = 22$$

Business impact: Reward design directly shapes what behavior an AI agent learns to repeat.



Try It Yourself

Using the reward table above, what's the total reward for 3 correct retrievals and 2 incorrect ones?

Answer: $3(15) - 2(8) = 45 - 16 = 29$.



AI Application Mapping

WHERE THIS SHOWS UP IN AI



Customer Service Agents

Choosing the best next action in a conversation



Workflow Automation

Selecting optimal task sequences



Resource Allocation

Assigning compute, inventory, or staff optimally



Dynamic Pricing

Optimizing price points for expected revenue

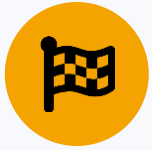


Hands-on Practice — Optimization

HANDS-ON MATHEMATICAL REASONING

Q. A delivery-routing agent scores 3 possible routes by expected efficiency: Route A = 0.75, Route B = 0.88, Route C = 0.62. Which route should the agent choose, and why?

Answer: Route B (0.88) — an optimization-driven agent always selects the action with the highest utility score (argmax).



Key Takeaways

- ✓ Probability & Bayes' Theorem let AI reason under uncertainty
- ✓ Vectors & embeddings let AI represent meaning as numbers
- ✓ Matrices & tensors power transformer operations in every modern LLM
- ✓ Token probabilities are literally how LLMs generate text
- ✓ Gradients are how AI models learn from their mistakes
- ✓ Cosine similarity powers semantic search and RAG retrieval
- ✓ Optimization & reward scores drive AI agent decision-making

Thank You